CSA06 – Design and Analysis of Algorithms

CO1 Assessment Tool 1: IT Application-Based Problem Solving

Scalability Audit of the ShopSphere E-Commerce Platform

Student Name: K.Hameed Basha

Registration Number:192425216

Course: CSA0610 – Design and Analysis of Algorithms

Assumptions

- The server executes 10^8 basic operations per second.
- For recurrence-based numeric estimates, the standard asymptotic operation-count model is used and, where an exact expansion is shown, $T(1) = 1$.
- The question does not specify base cases for M4 and M5, so their numeric counts are estimates, not exact measured runtimes.

Tasks Covered

- (a) Identify the input size, dominant basic operation and growth rate of each module.
- (b) Count M1's pairwise comparisons and M2's audit operations; test M2 at n = 1,024.
- (c) Solve the binary-search recurrence by backward substitution and compare iterative and recursive space.
- (d) Apply the Master Theorem to M4 and solve M5's recurrence.
- (e) Compare best, average and worst cases for sequential and binary search, including a probability-based average for S1.
- (f) Prove the report-generator function is $\Theta(n^2)$ and that it is not $O(n)$.
- (g) Estimate operations and execution times at n = 1,000,000, compare with the 10-minute window, and recommend redesigns.

(a) Understand the Problem

Let n be the number of products or records being processed.

Module	Input size	Dominant basic operation	Where it occurs	Expected growth
M1 – Duplicate-listing detector	Number of products (n)	Comparing two product titles and sellers	Inside both nested loops	$\Theta(n^2)$
M2 – API rate limiter	Number of requests (n)	Audit(Request[j])	Inner loop; outer index doubles	$\Theta(n)$
M3 – Product search S1	Number of catalogue entries (n)	Comparing the search key with one product	Sequential scan	Best $\Theta(1)$; average/worst $\Theta(n)$
M3 – Product search S2	Number of sorted index entries (n)	Comparing the key with the middle entry	Each step discards half the search range	$\Theta(\log n)$
M4 – Recommendation engine (current)	Data size (n)	Processing/merging subproblem results	Four subproblems of size n/2, plus linear merge	$\Theta(n^2)$
M4 – Recommendation engine (redesign)	Data size (n)	Processing/merging subproblem results	Two subproblems of size n/2, plus linear merge	$\Theta(n \log n)$
M5 – Sales analytics	Number of sales records (n)	Sorting/processing at each recursive level	Two half-size subproblems and n log n work per level	$\Theta(n \log^2 n)$

The report generator has $f(n) = 3n^2 + 10n \log_2 n + 50$. Its dominant term is $3n^2$, so its growth is $\Theta(n^2)$.

(b) Analyse the Iterative Modules

M1 – Duplicate-listing detector

The outer loop runs for i = 0, 1, …, n − 2. For each i, the inner loop compares product i with every later product, giving (n − i − 1) comparisons.

$C(n) = \Sigma\ (n - i - 1)$, summed for i = 0 to n − 2

Re-indexing the sum:

$C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1$

Using $1 + 2 + \ldots + m = m(m + 1)/2$ with m = n − 1:

$C(n) = n(n - 1)/2$

Therefore, the time complexity is **Θ(n^{2})**. The loop counters need **Θ(1)** auxiliary space, excluding any storage used by MarkDuplicate to record the duplicate results.

M2 – API rate limiter

The variable i takes the values 1, 2, 4, 8, … while i < n. For each value of i, the inner loop executes i audits. Let $m = \lceil \log_2 n \rceil$. The values are 2^0 through 2^{m-1}, so:

$A(n) = 1 + 2 + 4 + \ldots + 2^{m-1}$

By the geometric-series formula:

$A(n) = (2^m - 1)/(2 - 1)$

$A(n) = 2^{\lceil \log_2 n \rceil} - 1$

Since $n \leq 2^{\lceil \log_2 n \rceil} < 2n$ for n > 1, the operation count is linear: **Θ(n)**. The team member's Θ(n log n) claim is incorrect: the inner loop does not run n times on every outer iteration; its sizes are geometric and sum to less than 2n.

Test for n = 1,024 = 2^{10}: the loop values are 1, 2, 4, …, 512.

A(1024) = 1 + 2 + … + 512 = $2^{10} - 1$ = 1,023 audits

(c) Solve S2 by Backward Substitution

Given T(n) = T(n/2) + 1 with T(1) = 1, assume $n = 2^k$.

T(n) = T(n/2) + 1

Substitute again:

$T(n) = T(n/4) + 1 + 1 = T(n/4) + 2$

Substitute again:

$T(n) = T(n/8) + 3$

After k substitutions:

$T(n) = T(n/2^k) + k$

Since $n = 2^k$, we have $n/2^k = 1$ and $k = \log_2 n$. Hence:

$T(n) = T(1) + \log_2 n = 1 + \log_2 n$

$T(n) = \log_2 n + 1 = \Theta(\log n)$

Space complexity

- Iterative binary search uses **$\Theta(1)$** auxiliary space because it uses a fixed number of variables (low, high, mid).
- Recursive binary search uses **$\Theta(\log n)$** stack space because the recursion depth is logarithmic.

(d) Apply the Master Theorem

The Master Theorem form is $T(n) = aT(n/b) + f(n)$. We compare $f(n)$ with $n^{\log_b a}$.

M4 current design: $T(n) = 4T(n/2) + n$

- $a = 4$, $b = 2$, $f(n) = n$
- $n^{\log_b a} = n^{\log_2 4} = n^2$
- $f(n) = n = O(n^{2-1})$, which is polynomially smaller than n^2.

Master Theorem Case 1 applies:

$T(n) = \Theta(n^2)$

M4 proposed redesign: $T(n) = 2T(n/2) + n$

- $a = 2$, $b = 2$, $f(n) = n$
- $n^{\log_b a} = n^{\log_2 2} = n$
- $f(n) = \Theta(n^{\log_b a})$

Master Theorem Case 2 applies:

$T(n) = \Theta(n \log n)$

When the input doubles, the current quadratic design takes approximately four times as long. The redesigned n log n design takes approximately twice as long, with a small additional logarithmic factor.

M5: $T(n) = 2T(n/2) + n \log n$

- $a = 2$, $b = 2$, $f(n) = n \log n$
- $n^{\log_b a} = n^{\log_2 2} = n$
- $f(n) = \Theta(n \log^1 n)$

The three basic Master Theorem cases do not directly cover the logarithmic factor in this form, so we use the extended Master Theorem (generalised Case 2) or a recursion tree. The extended result is:

$T(n) = \Theta(n \log^{1+1} n)$

$T(n) = \Theta(n \log^2 n)$

Recursion-tree check for $n = 2^k$: at level i there are 2^i subproblems of size $n/2^i$. The work at that level is $2^i \cdot (n/2^i) \cdot \log_2(n/2^i) = n(k - i)$. Summing over all levels:

$n(k + (k - 1) + \ldots + 1) = n \cdot k(k + 1)/2 = \Theta(n \log^2 n)$

(e) Best, Average and Worst Search Cases

S1 – Sequential search

- **Best case:** the product is the first item. Comparisons = 1.
- **Average case** (product present, each position equally likely): $(1 + 2 + \ldots + n)/n = (n + 1)/2$ comparisons.
- **Worst case:** the product is last or absent. Comparisons = n.

If the product exists with probability p, and its position is uniformly distributed when present, then:

$E[C] = p \cdot (n + 1)/2 + (1 - p) \cdot n$

For $n = 1{,}000{,}000$ and $p = 0.5$:

$E[C] = 0.5 \times (1{,}000{,}001/2) + 0.5 \times 1{,}000{,}000$

$E[C] = 250{,}000.25 + 500{,}000 = 750{,}000.25$ comparisons on average

S2 – Binary search

- **Best case:** the key equals the first middle entry checked. Comparisons = 1.
- **Average case:** $\Theta(\log n)$ comparisons for a typical successful or unsuccessful search in a balanced sorted index.
- **Worst case:** the range is halved repeatedly until the key is found or the range is empty. For n = 1,000,000 this is about 20 comparisons, since $\lceil \log_2 1{,}000{,}000 \rceil = 20$.

Binary search requires the catalogue/index to be sorted and maintained; it is not a direct replacement for sequential search on an unsorted list.

(f) Prove $f(n) = \Theta(n^2)$ and $f(n) \notin O(n)$

Given:

$f(n) = 3n^2 + 10n \log_2 n + 50$

Lower bound

For $n \geq 1$, all terms are non-negative, so:

$f(n) \geq 3n^2$

Upper bound

For $n \geq 1$, $\log_2 n \leq n$ and $1 \leq n^2$. Therefore:

$f(n) = 3n^2 + 10n \log_2 n + 50 \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$

Thus, for every $n \geq 1$:

$3n^2 \leq f(n) \leq 63n^2$

Choose $c_1 = 3$, $c_2 = 63$ and $n_0 = 1$. By the formal definition of Θ:

$f(n) = \Theta(n^2)$

Proof that $f(n) \notin O(n)$

Since $f(n) \geq 3n^2$, we have:

$f(n)/n \geq 3n \rightarrow \infty$ as $n \rightarrow \infty$

No constant C can satisfy $f(n) \leq C \cdot n$ for all sufficiently large n. Hence:

$f(n) \notin O(n)$

(g) Scalability Decision for n = 1,000,000

The server speed is 10^8 basic operations per second. The 10-minute limit is 600 seconds, which is equivalent to 6×10^{10} basic operations.

Module / design	Operation-count estimate	Estimated time	Within 10 minutes?
M1 – Pairwise comparisons	$n(n-1)/2 = 499{,}999{,}500{,}000$	4,999.995 s ≈ 83.33 min	No
M2 – Rate limiter	$2^{\lceil \log_2 n \rceil} - 1 = 1{,}048{,}575$	0.0105 s	Yes
S2 – Binary search	About 20 comparisons per search	2.0×10^{-7} s per search	Yes
M4 current – $4T(n/2) + n$	Approx. $2n^2 - n$ (assuming $T(1) = 1$, power-of-two expansion)	Approx. 20,000 s ≈ 333.33 min	No
M4 redesign – $2T(n/2) + n$	Approx. $n \log_2 n + n$	Approx. 0.209 s	Yes
M5 – $2T(n/2) + n \log n$	Approx. $n[1 + k(k+1)/2]$, $k = \log_2 n$, $T(1) = 1$	Approx. 2.10 s	Yes

Note: the M4 and M5 estimates use the stated assumptions with $k \approx \log_2(1{,}000{,}000) \approx 19.93$. Actual elapsed time depends on implementation, hardware, I/O, constants, and whether the recurrence's base-case cost differs. For M2, the exact formula gives $2^{20} - 1 = 1{,}048{,}575$ because 1,000,000 is not a power of two.

Recommendations

- **Redesign M1:** replace all-pairs comparison with a hash-based grouping/index on (seller, normalized_title). Expected time can become $\Theta(n)$ with hashing instead of $\Theta(n^2)$. Verify normalisation rules and hash collisions.

- **Redesign M4:** remove the unnecessary replication as proposed. The recurrence improves from $\Theta(n^2)$ to $\Theta(n \log n)$, reducing the estimated work substantially.
- **Keep M2's doubling strategy:** its total audit count is $\Theta(n)$, not $\Theta(n \log n)$.
- **Use S2 for catalogue search:** maintain a sorted index and use binary search for $\Theta(\log n)$ lookup. If frequent updates make sorting expensive, consider a suitable database index.
- **Review M5 only if production measurements show a bottleneck:** its $\Theta(n \log^2 n)$ estimate is below the 10-minute operation budget under the simplified model, but profiling is still needed.
- **Report generator:** its $\Theta(n^2)$ cost is also a scalability risk if run over the full catalogue at the stated operation rate.

Conclusion

M1 and the current M4 design exceed the 10-minute window in the simplified model and should be redesigned. The report generator is also quadratic and should be reviewed. M2 and S2 scale well; the proposed M4 design and M5 are estimated to fit within the window.